

大作业报告：CdeBattle编程元学习工具

1. 程序功能介绍

本程序是一款基于 Qt 框架开发的桌面端答题 RPG 游戏，将编程知识学习与回合制战斗相结合，通过游戏化的方式提升学习兴趣与编程能力，主要包含以下功能模块：

- 主菜单：提供“学习模式”、“竞技模式”、“错题本”、“成就展墙”、“排行榜”、“帮助”和“退出”按钮，提供入口。

- 学习模式：第0章是机制展示，用于演示战斗机制，其他关卡为正常编程学习关卡，战前可自定义难度权重（简单/中等/困难）和题型权重（选择题/代码补全题），满足奖杯条件可解锁成就，战斗中每回合随机抽题，答对则对敌人造成伤害，答错则受敌人反击。每3回合触发一次额外回合，可从随机技能中选择一个并自选难度，该难度对应下一题的难度，答对后获得攻击力、防御力、血量上线、吸血比例等永久属性提升，支持存档与读档，战斗结束后若失败可选择直接重新开始、修改配置后重开或返回主菜单，会记录总伤害、答对题数等数据并更新排行榜。

- 竞技模式：实时无尽战斗模式。敌人拥有无限生命且属性随时间自动增长，并每隔一段时间自动攻击玩家。每轮玩家先选择技能和难度，然后解答题目，题目都是代码补全题，可查看代码片段与示例，支持一键复制代码，复制样例输入输出。答对获得属性加成并攻击敌人，答错积累容错，容错耗尽后受到敌人攻击。使用“Pass卡”可以跳过当前题目。生命值归零后游戏结束，根据总伤害生成排行榜记录，并检测是否解锁竞技类成就。

- 错题本：自动收集学习模式和竞技模式中答错的题目。支持按章节

筛选、重做错题（弹出独立答题窗口）、单条删除和多选批量删除，帮助针对性复习。

·成就展墙：展示游戏内的所有成就。可按“学习”或“战斗”类别进行筛选，已解锁的成就高亮显示。

·排行榜：记录历史战斗记录，按单局总伤害降序排列。

·帮助：介绍学习模式和竞技模式的玩法规则、操作方式。

2. 项目各模块与类设计细节

2.1 页面管理与导航

·UIManager:使用枚举Page管理所有界面，通过 QMap<Page, QWidget*> 持有页面实例，QList<Page> 维护导航历史。提供 goTo(Page) 进行前进，goBack() 返回上一页。负责在切换至战斗页时传递参数（章节ID、难度配置、是否读档）。

2.2 页面类

·MainMenuPage: 连接各按钮信号，调用 UIManager::goTo() 实现页面跳转。点击“竞技模式”时，会在跳转前将 BattlePage 设置为竞技模式。

·ChapterSelectPage: 通过 for 循环动态生成9个章节按钮，按钮文本通过读取对应章节 JSON 文件获取。点击章节时检查存档，若无存档则弹出showConfig Dialogue()配置权重；若有存档则提供“新游戏”与“读档”选项。配置对话框采用两步向导形式（第一步设置难度权重，第二步设置题型权重），验证输入后传递给 ·UIManager。

·BattlePage:回合制逻辑和竞技模式逻辑

Stats m_playerStats, m_enemyStats : 玩家和敌人属性（HP、攻击、防御、吸血等）。

- `QuestionBank m_questionBank`：题库管理器。
- `SaveManager m_saveManager`：存档管理。
- `WrongBookManager m_wrongBookManager`：错题本管理。
- `AchievementManager m_achievementManager`：成就管理。
- `RankingManager m_rankingManager`：排行榜管理。
- `QMap<int, QSet<int>> m_usedQuestionIds`：已用题目ID（避免重复出题）。
- `QMap<int, CodeCompletionQuestion> m_completionQuestions`：预加载的代码补全题。
- 竞技模式专属：定时器 `m_arenaTimer`、倒计时条 `m_countdownBar`、Pass按钮 `m_passBtn`、状态机 `m_arenaState` 等。
- 动画相关：精灵（`m_enemySprite`, `m_playerSprite`）的帧动画和状态切换。

关键方法：

- `showNextQuestion()`：根据权重抽选题目，优先抽取代码补全题（若题型权重满足），选择题和代码补全题均支持重抽与去重。
- `onAnswerSubmitted()`：处理答题反馈，区分普通回合和额外回合。学习模式下答对攻击并吸血，答错受击；竞技模式下答对加属性并攻击，答错扣容错。
- `onExtraRoundTrigger()`：触发额外回合，生成技能列表，调用 `SkillPanel` 展示。
- `onSkillSelected()`：根据玩家选择的技能和难度记录待应用增益，并抽取相应题目。
- `resetBattle()` / `loadFromArchive()`：重置或加载存档状态。
- `onBattleFinished()` / `onArenaBattleFinished()`：结算战斗结果，更新排行榜和成就，弹出结果对话框。

·`ResultPage`：显示战斗结果（胜利/失败）、总伤害、回合数、答对题数、奖杯状态，并展示本章最佳历史记录。

·`ErrorBookPage`：从 `WrongBookManager` 加载错题列表，使用 `QListWidget` 展示，支持点击查看解析、重做、删除、全选/批量删除。重做功能通过内嵌 `QuestionWidget` 的对话框实现。

·AchievementPage: 加载成就定义并显示成就列表, 通过QComboBox按类别筛选, 列表项根据解锁状态显示不同底色。

·RankingPage:从 RankingManager 获取所有竞技记录, 按总伤害降序排列。

·SettingsPage: 使用 QTextEdit 展示游戏规则说明。

2.3 自定义组件

·QuestionWidget: 使用 QStackedWidget 管理题目面板: 选择题 (QLabel 模拟按钮, 通过事件过滤器实现点击高亮和提交)、代码补全题 (带代码模板弹窗和文本编辑器)。支持提交后视觉反馈 (颜色闪烁) 和容错次数提示。

·SkillPanel: 动态生成技能按钮, 学习模式下点击按钮弹出QMenu选择难度; 竞技模式下直接根据预设难度发出信号。面板显示与隐藏均带有透明度渐变动画。

·StatusBar: 使用 QProgressBar 和 QLabel 显示双方面板, 包含暂停按钮 (发射 pauseClicked 信号), 在竞技模式下自动切换为显示倒计时条、Pass按钮和总伤害。

2.4 数据管理器

·QuestionBank: 负责加载 JSON 题库文件, 使用 QMap<int, QVector<Question>> 按章节存储题目, drawQuestion(chapterId, difficulty, usedIds) 随机抽题 (支持难度回退策略)。

·SaveManager: 管理章节存档的读写删除, 使用 ChapterArchive 结构体序列化/反序列化为 JSON 文件。

·WrongBookManager: 管理错题条目 (WrongBookEntry), 支持按章

节筛选、添加（合并错误次数）、删除条目，数据持久化到本地JSON。

·**AchievementManager**：加载成就定义（JSON）和进度文件，提供 unlockAchievement(id) 和按类别筛选。成就条件检查分散在各页面。

·**RankingManager**：维护排行榜（QVector<RankingEntry>），支持添加记录、获取最高记录、按总伤害排序和持久化。

2.5 数据模型

·**Question / CodeCompletionQuestion**：题目实体，包含题目文本、选项、答案、难度、代码模板、测试用例等。

- **ChapterArchive**：存档结构，包含双方属性、已用题目ID、回合数、难度配置。

- **Stats**：角色属性结构体，提供 toJson() / fromJson() 序列化。

- **GameConfig**：难度和题型权重配置，包含随机抽选方法和奖杯阈值判断。

2.6 工具类

·**JsonUtils**：封装 QJsonDocument 的读写操作。

- **CodeJudge**：用于代码补全题的编译运行与结果比对（调用游戏自身打包的编译器），返回通过/失败及错误信息。

- **RandomUtils**：基于 QRandomGenerator 的加权随机函数。

3. 小组成员分工情况：

齐舞飞扬：编写战斗回合逻辑，判题逻辑，战斗数值，难度和权重分配逻辑，排行榜，帮助，错题本 MultiSelection 多选 + 全选 + 批量删除，代码补全题接入 CodeJudge，帮助页面布局优化，进行错误的检查与修复。

李笑天：设计题库数据结构，设计错题本界面，成就界面，构建题库成就，记录游戏数据。

邹宇翔：UI设计，界面整体框架搭建，角色动画，大作业报告编写以及录制演示视频。

4. 项目总结与反思

项目基于 Qt 5.15 + C++17 开发，利用了 Qt 的信号槽机制，属性动画系统、JSON 支持和文件 I/O。

项目亮点：

- **较丰富的游戏化元素**：回合制战斗、额外回合、竞技模式、成就系统、排行榜等。
- **灵活的出题机制**：支持权重配置、题型混合、已用题目去重与自动重置，保证了题目的多样性和可定制性。
- **完善的存档系统**：学习模式支持存档和读档，存档包含完整的战斗状态和配置，可无缝恢复。
- **像素风 UI 与动画**：自定义精灵帧动画、浮动伤害文字、页面滑动切换、透明度渐变等提升了视觉表现。

不足与改进方向：

- **BattlePage 过于臃肿**：战斗逻辑、动画控制、存档、成就检测等全部耦合在一个类中，未来可拆分为 BattleLogic、ArenaLogic、Animator 等模块。
- **界面可配置性**：大量样式硬编码在 C++ 中，后续可考虑使用

QSS 文件统一管理，便于主题切换。

收获：

通过本项目的开发，深入理解了 Qt 事件循环、信号槽跨线程安全性、属性动画的应用，。在游戏逻辑与 UI 交互的同步、状态管理以及资源打包部署等方面积累了宝贵经验。同时认识到大型项目架构规划的重要性，以及持续重构对代码质量的提升作用。学习到了大型程序设计的一般流程与特定技能，为进一步的知识学习打下了坚实的基础。